PYTHON ACTIVITY BASED LEARNING

Case Study Report

Name	**G. Sai Sri Durga**
Section	**4**
Roll No.	**26001075**

INTRODUCTION

This report presents ten Python Activity Based Learning case studies selected from the assigned list of 100. Each case is organized with its problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python program, test cases, output screenshot, and conclusion. All programs use only the assigned syllabus: variables, data types, operators, conditional statements, loops, strings, lists, nested lists, tuples and tuple unpacking.

CONTENTS — TEN CASE STUDIES

Case 02. Progressive Electricity Billing

Case 13. ATM Transaction Sequence Simulation

Case 42. Student Marks Matrix Analysis

Case 49. Tuple-Based Product Catalogue Audit

Case 53. Food Waste Monitoring System

Case 57. Smart Canteen Order and Stock Management

Case 66. Student Academic Risk Classification

Case 77. Smart Electricity Billing for Multiple Apartments

Case 87. Examination Question Difficulty and Performance Analysis

Case 91. Advanced String-Based Registration Record Analyzer

CASE 1: Progressive Electricity Billing

1. Problem Statement

Develop a Python program that calculates household electricity bills using progressive consumption slabs and a fixed monthly charge. The bills of several months are calculated in sequence so that they can be compared easily.

2. Objectives

- Apply different unit rates to different consumption slabs.
- Add a fixed monthly charge to the energy charge.
- Calculate and display the bill for each month's consumption.
- Allow the bills of several months to be compared.

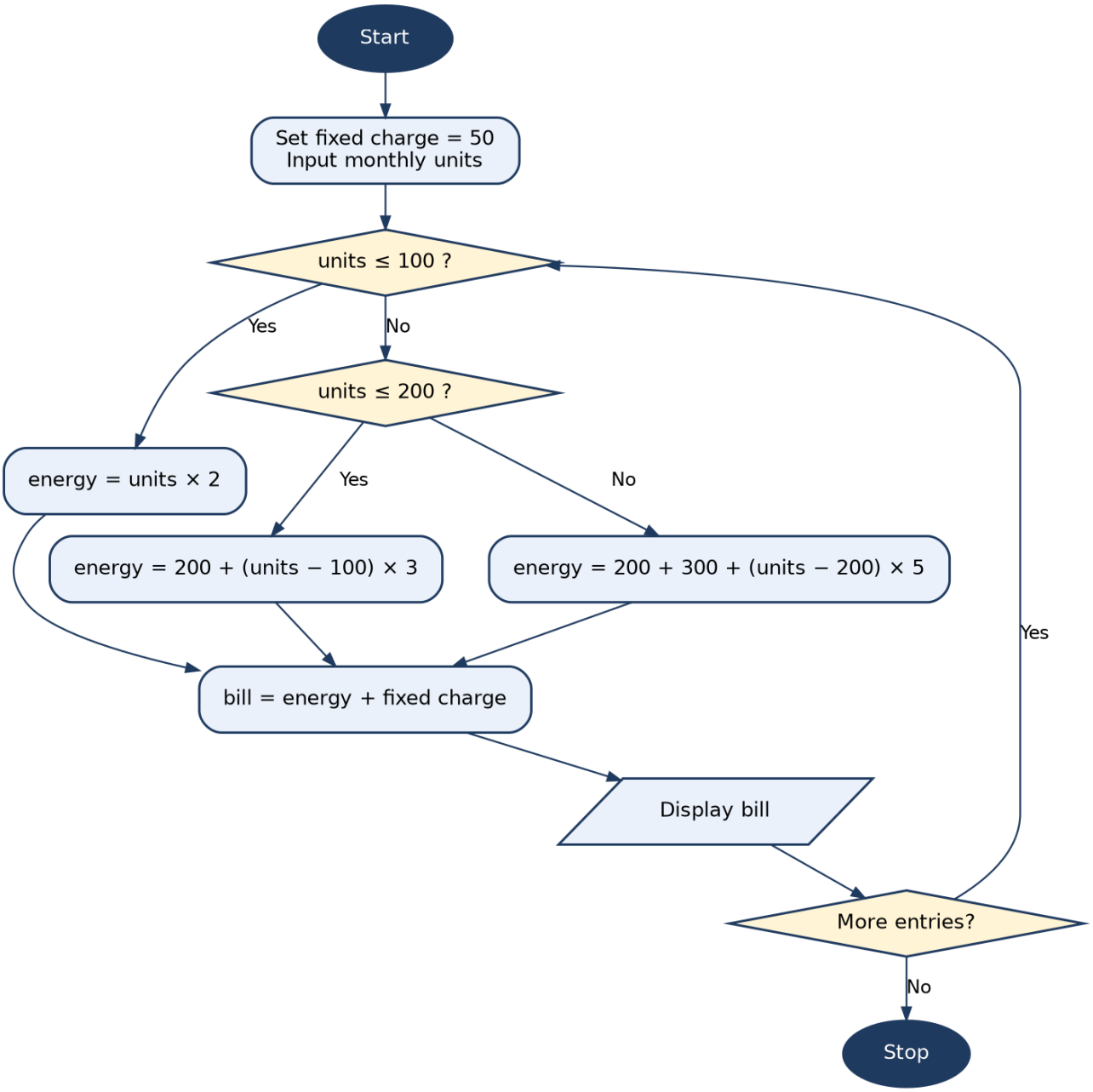
3. Assumptions

- These are sample tariff rules for the program; they are not official rates.
- Slab 1: first 100 units at ₹2 per unit.
- Slab 2: units 101 to 200 at ₹3 per unit.
- Slab 3: units above 200 at ₹5 per unit.
- A fixed charge of ₹50 is added to every bill.
- Units consumed are whole, non-negative numbers.

4. Algorithm

1. Start.
2. Set the fixed charge to 50 and store the monthly units in a list.
3. Take the next monthly consumption value.
4. If units ≤ 100, energy charge = units × 2.
5. Else if units ≤ 200, energy charge = 200 + (units − 100) × 3.
6. Else energy charge = 200 + 300 + (units − 200) × 5.
7. Bill = energy charge + fixed charge; display the bill.
8. Repeat steps 3 to 7 for all months.
9. Stop.

5. Flowchart


Start
Set fixed charge = 50
Input monthly units
units ≤ 100 ?
Yes
No
units ≤ 200 ?
energy = units × 2
Yes
No
energy = 200 + (units − 100) × 3
energy = 200 + 300 + (units − 200) × 5
bill = energy + fixed charge
Display bill
More entries?
Yes
No
Stop

6. Pseudocode

```
START
SET fixed_charge = 50
STORE monthly units in a list

FOR each units in the list
    IF units <= 100 THEN
        energy = units * 2
    ELSE IF units <= 200 THEN
        energy = 200 + (units - 100) * 3
    ELSE
        energy = 200 + 300 + (units - 200) * 5
    END IF

    bill = energy + fixed_charge
    DISPLAY bill
END FOR

STOP
```

7. Python Source Code / Program

```
# Case 02: Progressive Electricity Billing
# Slabs: 0-100 units @ Rs.2, 101-200 units @ Rs.3, above 200 units @ Rs.5
# Fixed charge: Rs.50 per bill

fixed_charge = 50
monthly_units = [0, 50, 100, 150, 250]

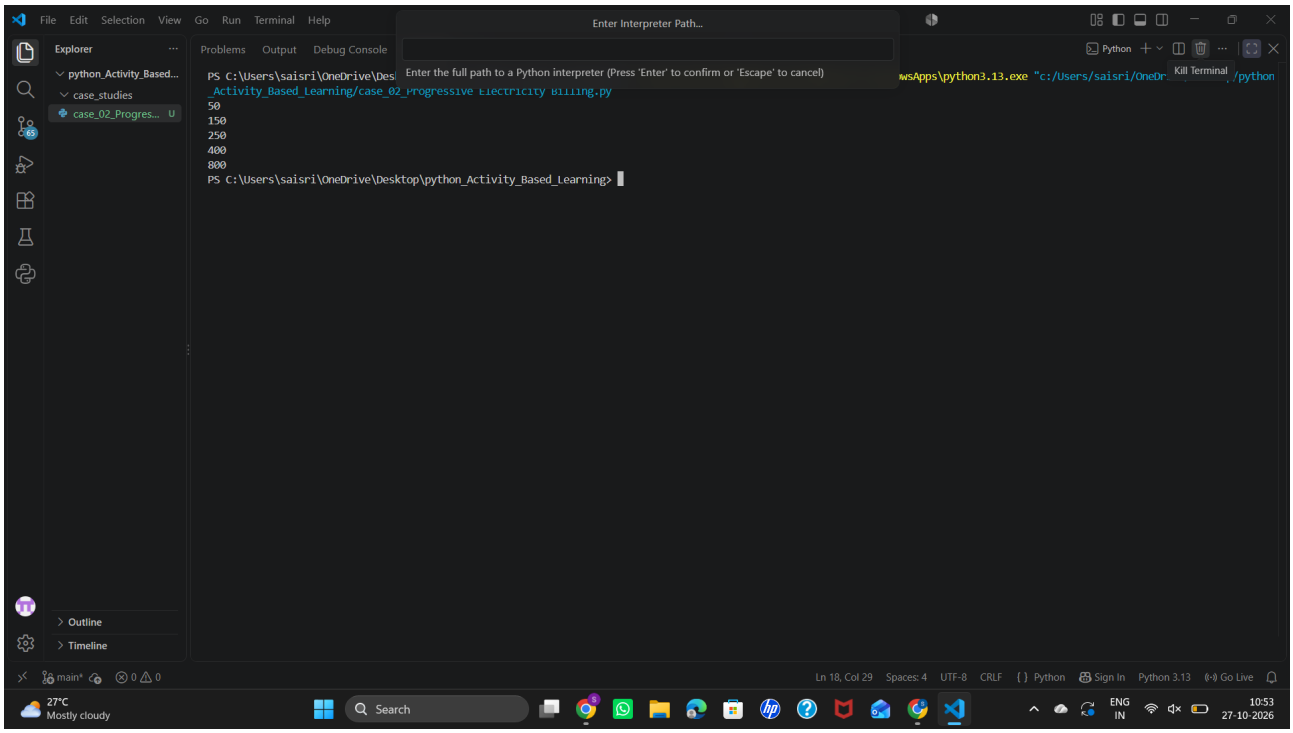
for units in monthly_units:
    if units <= 100:
        energy_charge = units * 2
    elif units <= 200:
        energy_charge = 100 * 2 + (units - 100) * 3
    else:
        energy_charge = 100 * 2 + 100 * 3 + (units - 200) * 5

    bill = energy_charge + fixed_charge
    print(bill)
```

8. Test Cases

Test	Input	Expected output
1	0 units	50
2	50 units	150
3	100 units	250
4	150 units	400
5	250 units	800

9. Output Screenshot



10. Conclusion

The program applies progressive slab rates correctly and adds the fixed charge. Higher consumption moves into costlier slabs, so the bill grows faster than the units, which is what a progressive tariff is meant to do.

CASE 2: ATM Transaction Sequence Simulation

1. Problem Statement

Develop a Python program that processes a sequence of deposit and withdrawal transactions on a simulated ATM account, keeps the running balance, rejects invalid transactions and reports the final balance.

2. Objectives

- Process a sequence of deposits and withdrawals using tuples.
- Maintain the running account balance.
- Enforce the withdrawal limit and reject invalid transactions.
- Display the final balance for each test case.

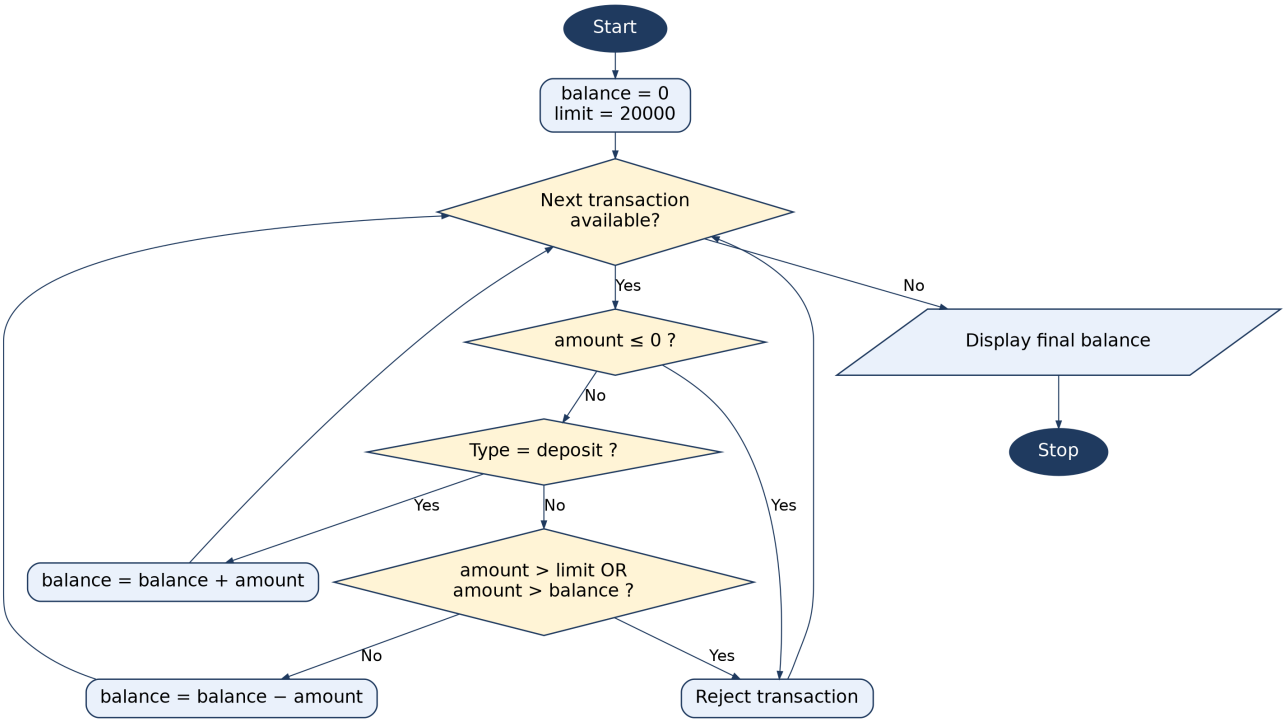
3. Assumptions

- The account starts every test case with a balance of ₹0.
- The maximum amount for one withdrawal is ₹20,000.
- A transaction with an amount of zero or less is invalid.
- A withdrawal larger than the current balance is rejected.
- Rejected transactions do not change the balance.

4. Algorithm

1. Start.
2. Set the withdrawal limit and store the test cases as lists of (type, amount) tuples.
3. For each test case set balance = 0.
4. Take the next transaction and unpack it into type and amount.
5. If amount ≤ 0, reject it.
6. If the type is deposit, add the amount to the balance.
7. If the type is withdraw, reject it when amount > limit or amount > balance; otherwise subtract it.
8. After the last transaction display the final balance.
9. Stop.

5. Flowchart



6. Pseudocode

```
START
SET withdrawal_limit = 20000

FOR each test case
    balance = 0
    FOR each (kind, amount) in the transactions
        IF amount <= 0 THEN
            REJECT transaction
        ELSE IF kind is "deposit" THEN
            balance = balance + amount
        ELSE IF kind is "withdraw" THEN
            IF amount > withdrawal_limit OR amount > balance THEN
                REJECT transaction
            ELSE
                balance = balance - amount
            END IF
        END IF
    END FOR
    DISPLAY balance
END FOR

STOP
```

7. Python Source Code / Program

```
# Case 13: ATM Transaction Sequence Simulation
# Each test case is a list of (transaction type, amount) tuples.

withdrawal_limit = 20000

test_cases = [
    [("deposit", 5000), ("withdraw", 1000)],
    [("deposit", 3000), ("withdraw", 500)],
    [("deposit", 1000), ("withdraw", 1000)],
    [("deposit", 2000), ("withdraw", 5000), ("withdraw", 1000)],
    [("deposit", 4000), ("withdraw", -500)]
]

case_number = 1
for transactions in test_cases:
    balance = 0

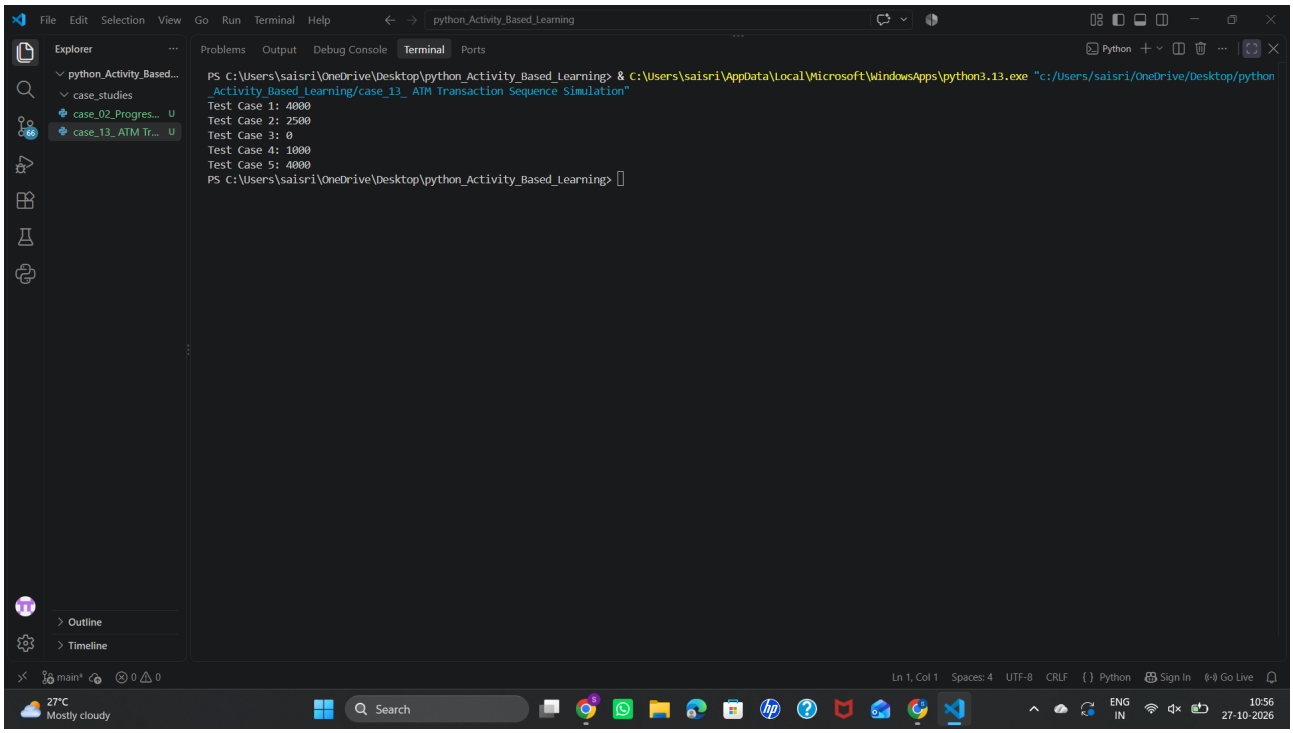
    for kind, amount in transactions:
        if amount <= 0:
            continue                    # invalid amount -> rejected
        if kind == "deposit":
            balance = balance + amount
        elif kind == "withdraw":
            if amount > withdrawal_limit:
                continue                # above limit -> rejected
            if amount > balance:
                continue                # insufficient balance -> rejected
            balance = balance - amount

    print("Test Case " + str(case_number) + ":", balance)
    case_number = case_number + 1
```

8. Test Cases

Test	Input	Expected output
1	Deposit 5000, Withdraw 1000	4000
2	Deposit 3000, Withdraw 500	2500
3	Deposit 1000, Withdraw 1000	0
4	Deposit 2000, Withdraw 5000 (rejected), Withdraw 1000	1000
5	Deposit 4000, Withdraw −500 (invalid)	4000

9. Output Screenshot



10. Conclusion

The program keeps the balance correct through a mixed sequence of transactions. Invalid amounts and over-limit or over-balance withdrawals are rejected without changing the balance.

CASE 3: Student Marks Matrix Analysis

1. Problem Statement

Develop a Python program that stores student marks as a matrix (nested list) with one row per student and one column per subject, and calculates the total and the average for every student.

2. Objectives

- Represent marks as a nested list.
- Use nested loops to visit every mark in the matrix.
- Calculate the total marks of each student.
- Calculate and display the average marks of each student.

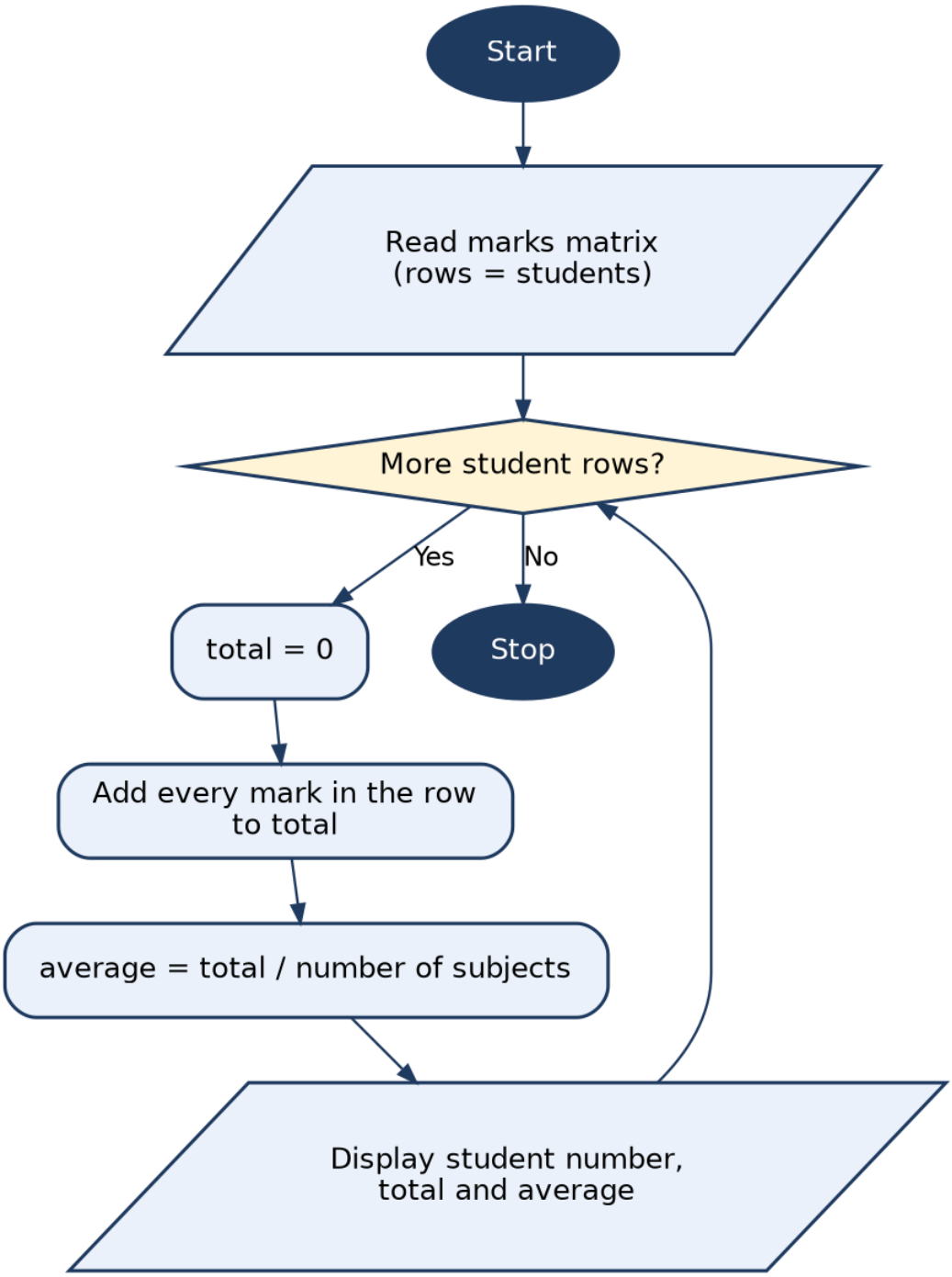
3. Assumptions

- Each row of the matrix holds the marks of one student.
- Every student has marks in three subjects.
- All marks are valid whole numbers from 0 to 100.
- The average is the total divided by the number of subjects.

4. Algorithm

1. Start.
2. Store each test case as a matrix of marks.
3. Take the next student row.
4. Set total = 0.
5. Add every mark of the row to total.
6. average = total ÷ number of subjects.
7. Display the student number, total and average.
8. Repeat for all rows and all test cases.
9. Stop.

5. Flowchart


Start
Read marks matrix
(rows = students)
More student rows?
Yes
No
total = 0
Stop
Add every mark in the row
to total
average = total / number of subjects
Display student number,
total and average

6. Pseudocode

```
START
STORE marks matrix (one row per student)

FOR each test case
    student_number = 1
    FOR each row in the matrix
        total = 0
        FOR each mark in the row
            total = total + mark
        END FOR
        average = total / number of marks
        DISPLAY student_number, total, average
        student_number = student_number + 1
    END FOR
END FOR

STOP
```

7. Python Source Code / Program

```
# Case 42: Student Marks Matrix Analysis
# Each test case is a matrix: one row per student, one column per subject.

test_cases = [
    [[75, 80, 85]],
    [[60, 70, 80]],
    [[90, 85, 95]],
    [[55, 60, 65]],
    [[40, 50, 60]]
]

case_number = 1
for matrix in test_cases:
    print("Test Case " + str(case_number) + ":")

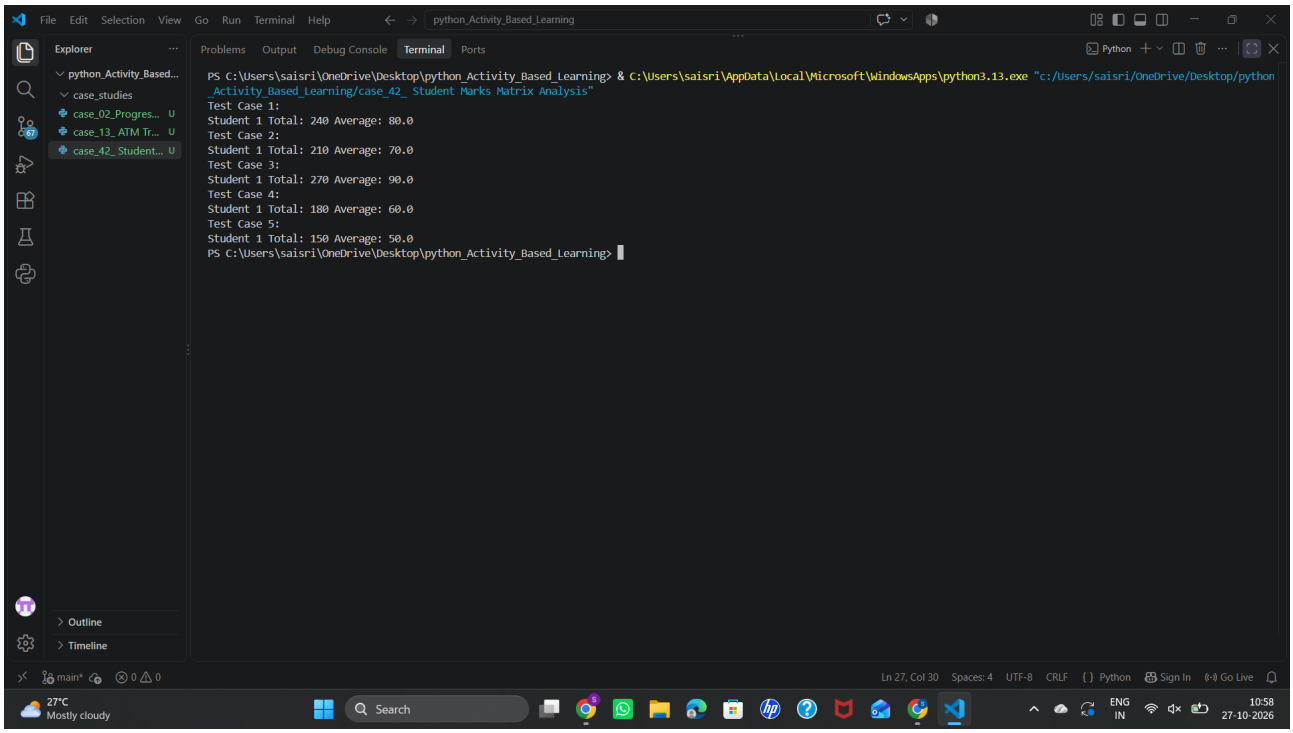
    student_number = 1
    for row in matrix:
        total = 0
        for mark in row:
            total = total + mark
        average = total / len(row)
        print("Student", student_number, "Total:", total, "Average:", average)
        student_number = student_number + 1

    case_number = case_number + 1
```

8. Test Cases

Test	Input	Expected output
1	[75, 80, 85]	Total: 240 Average: 80.0
2	[60, 70, 80]	Total: 210 Average: 70.0
3	[90, 85, 95]	Total: 270 Average: 90.0
4	[55, 60, 65]	Total: 180 Average: 60.0
5	[40, 50, 60]	Total: 150 Average: 50.0

9. Output Screenshot



10. Conclusion

Nested lists and nested loops make it simple to process a marks matrix. The totals and averages match the manual calculations for every test case.

CASE 4: Audit Tuple-Based Product Catalogue

1. Problem Statement

Develop a Python program that stores each product as a tuple of identifier, name, price and stock, detects invalid records and calculates the inventory value of every valid product.

2. Objectives

- Store product details in tuples.
- Use tuple unpacking to read the fields of each product.
- Detect invalid records.
- Calculate and display the total value of each product.

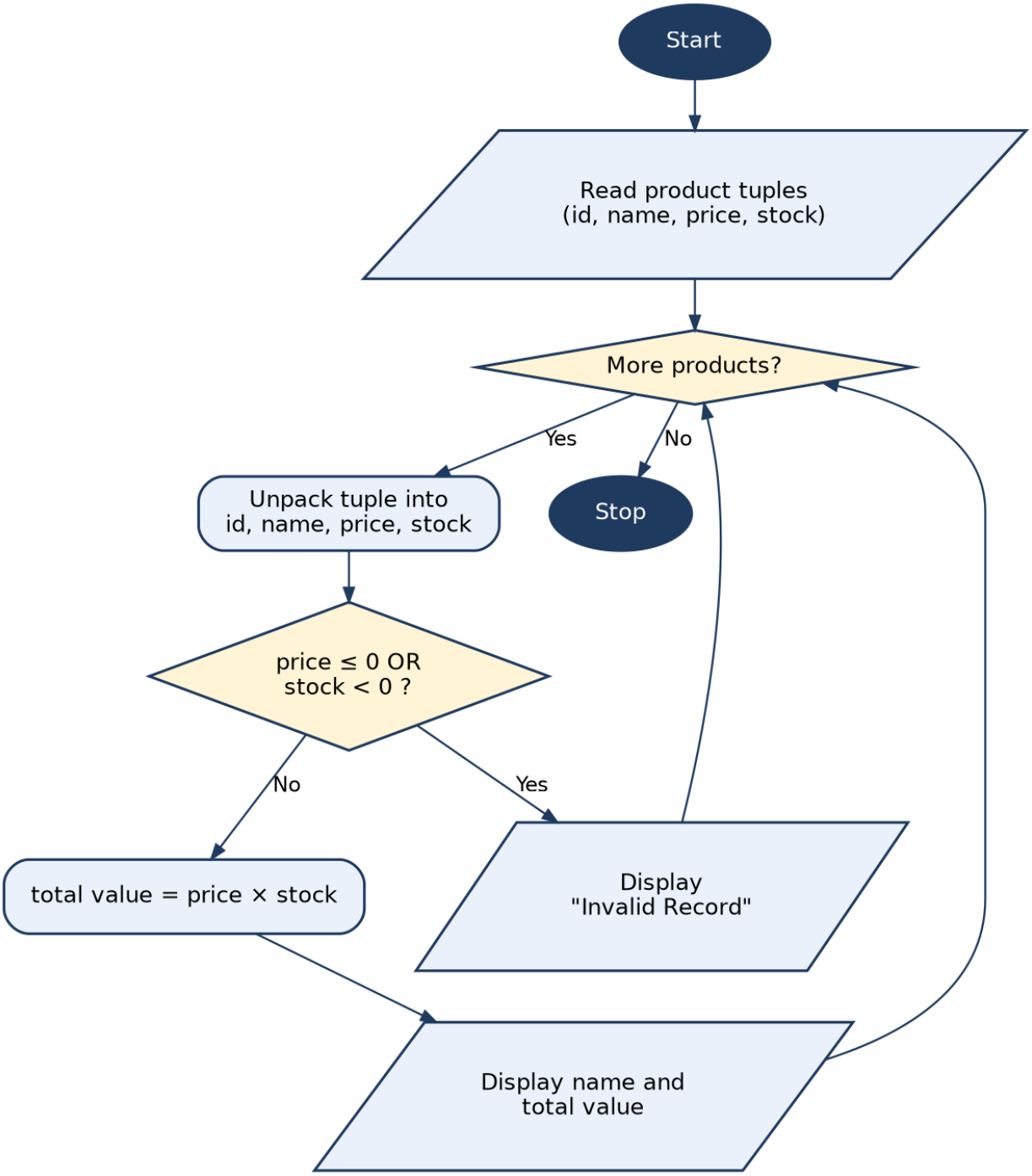
3. Assumptions

- Each product is a tuple: (product id, name, price, stock).
- A record is invalid if the price is zero or less, or the stock is negative.
- Total value = price × stock.
- Prices are in rupees.

4. Algorithm

1. Start.
2. Store the products as tuples in a list.
3. Take the next product and unpack it into id, name, price and stock.
4. If price ≤ 0 or stock < 0, display Invalid Record.
5. Otherwise total value = price × stock.
6. Display the product name and its total value.
7. Repeat for all products.
8. Stop.

5. Flowchart


Start
Read product tuples
(id, name, price, stock)
More products?
Yes
No
Stop
Unpack tuple into
id, name, price, stock
price ≤ 0 OR
stock < 0 ?
No
Yes
total value = price × stock
Display
"Invalid Record"
Display name and
total value

6. Pseudocode

```
START
STORE products as tuples (id, name, price, stock)

FOR each product
    UNPACK product into id, name, price, stock
    IF price <= 0 OR stock < 0 THEN
        DISPLAY name, "Invalid Record"
    ELSE
        total_value = price * stock
        DISPLAY name, total_value
    END IF
END FOR

STOP
```

7. Python Source Code / Program

```
# Case 49: Tuple-Based Product Catalogue Audit
# Each product is a tuple: (product id, name, price, stock)

test_cases = [
    ("P101", "Pen", 10, 5),
    ("B102", "Book", 100, 2),
    ("B103", "Bag", 500, 2),
    ("P104", "Pencil", 5, 10),
    ("B105", "Bottle", 150, 2)
]

case_number = 1
for product in test_cases:
    product_id, name, price, stock = product      # tuple unpacking

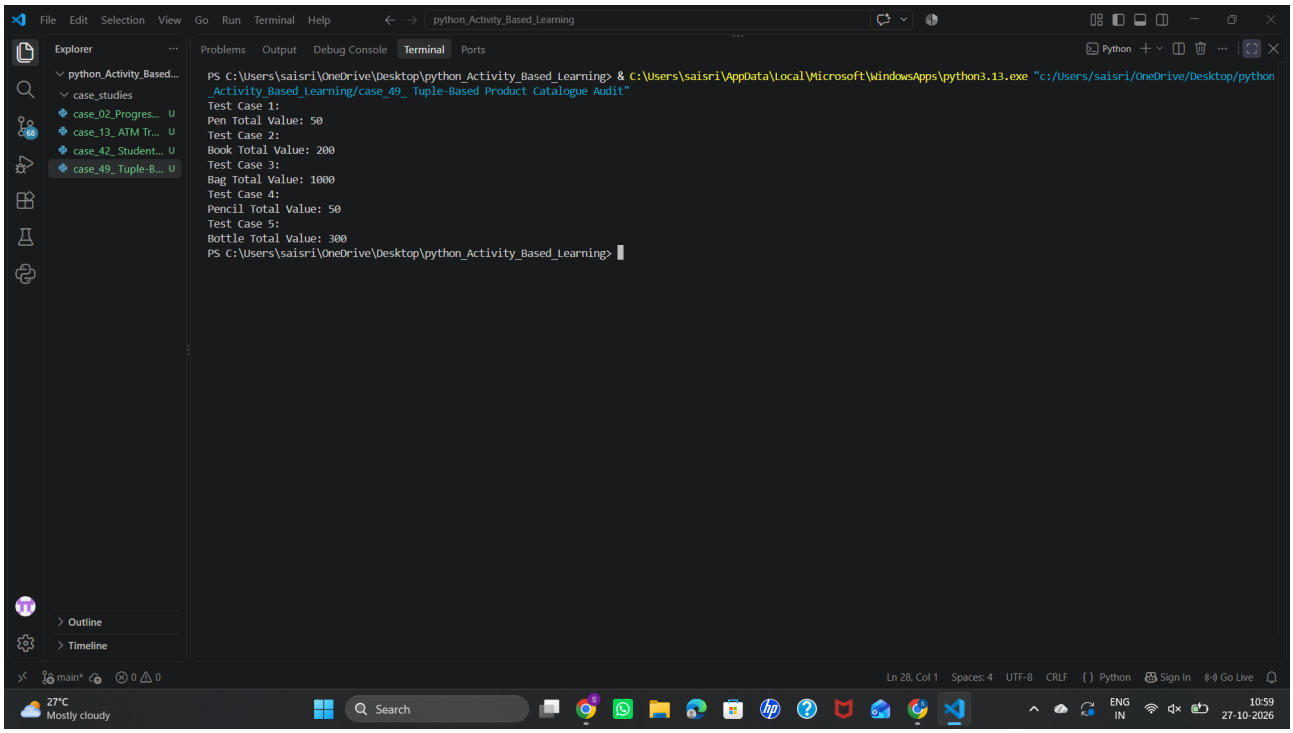
    print("Test Case " + str(case_number) + ":")
    if price <= 0 or stock < 0:
        print(name, "Invalid Record")
    else:
        total_value = price * stock
        print(name, "Total Value:", total_value)

    case_number = case_number + 1
```

8. Test Cases

Test	Input	Expected output
1	Pen, price 10, stock 5	Pen Total Value: 50
2	Book, price 100, stock 2	Book Total Value: 200
3	Bag, price 500, stock 2	Bag Total Value: 1000
4	Pencil, price 5, stock 10	Pencil Total Value: 50
5	Bottle, price 150, stock 2	Bottle Total Value: 300

9. Output Screenshot



10. Conclusion

Tuples keep each product's fields together, and unpacking makes the code easy to read. The program validates every record and gives the correct inventory value for each product.

CASE 5: Food Waste Monitoring System

1. Problem Statement

Develop a Python program that reads the food waste recorded over several days and calculates the total and the average food waste, so that wastage can be monitored.

2. Objectives

- Store the daily food waste in a list.
- Calculate the total food waste using a loop.
- Calculate the average daily food waste.
- Display the results in kilograms.

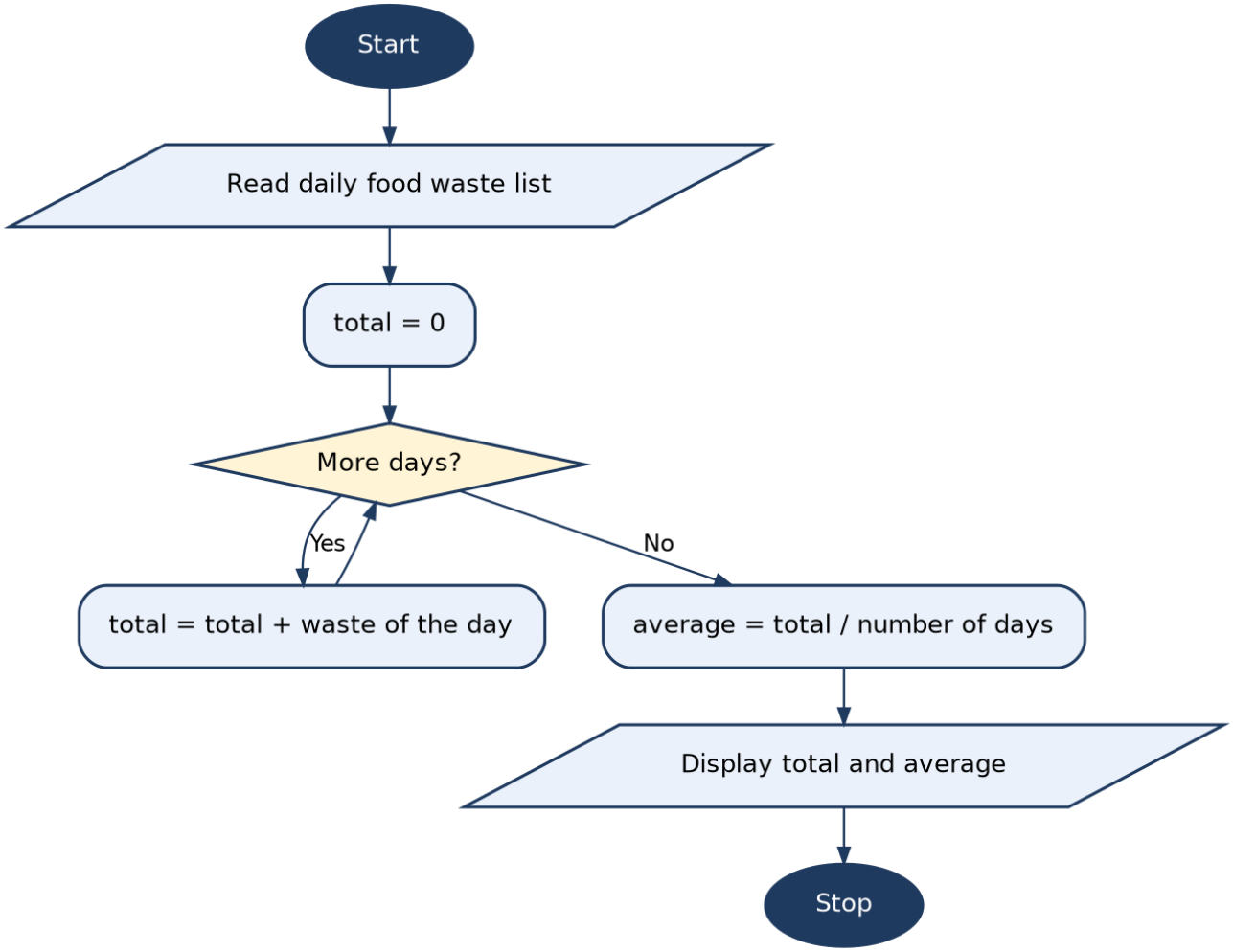
3. Assumptions

- Each test case holds the waste (in kg) for three days.
- Waste values are non-negative numbers.
- Average = total waste ÷ number of days.

4. Algorithm

1. Start.
2. Store each test case as a list of daily waste values.
3. Set total = 0.
4. Add each day's waste to total using a loop.
5. average = total ÷ number of days.
6. Display the total and the average in kg.
7. Repeat for all test cases.
8. Stop.

5. Flowchart



6. Pseudocode

```
START
STORE daily waste values in lists

FOR each test case
    total = 0
    FOR each waste in the list
        total = total + waste
    END FOR
    average = total / number of days
    DISPLAY total, "kg"
    DISPLAY average, "kg"
END FOR

STOP
```

7. Python Source Code / Program

```
# Case 53: Food Waste Monitoring System
# Each test case holds the food waste (kg) recorded on three days.

test_cases = [
    [15, 20, 25],
    [5, 10, 15],
    [2, 4, 6],
    [10, 20, 30],
    [8, 10, 12]
]

case_number = 1
for waste_list in test_cases:
    total_waste = 0
    for waste in waste_list:
        total_waste = total_waste + waste

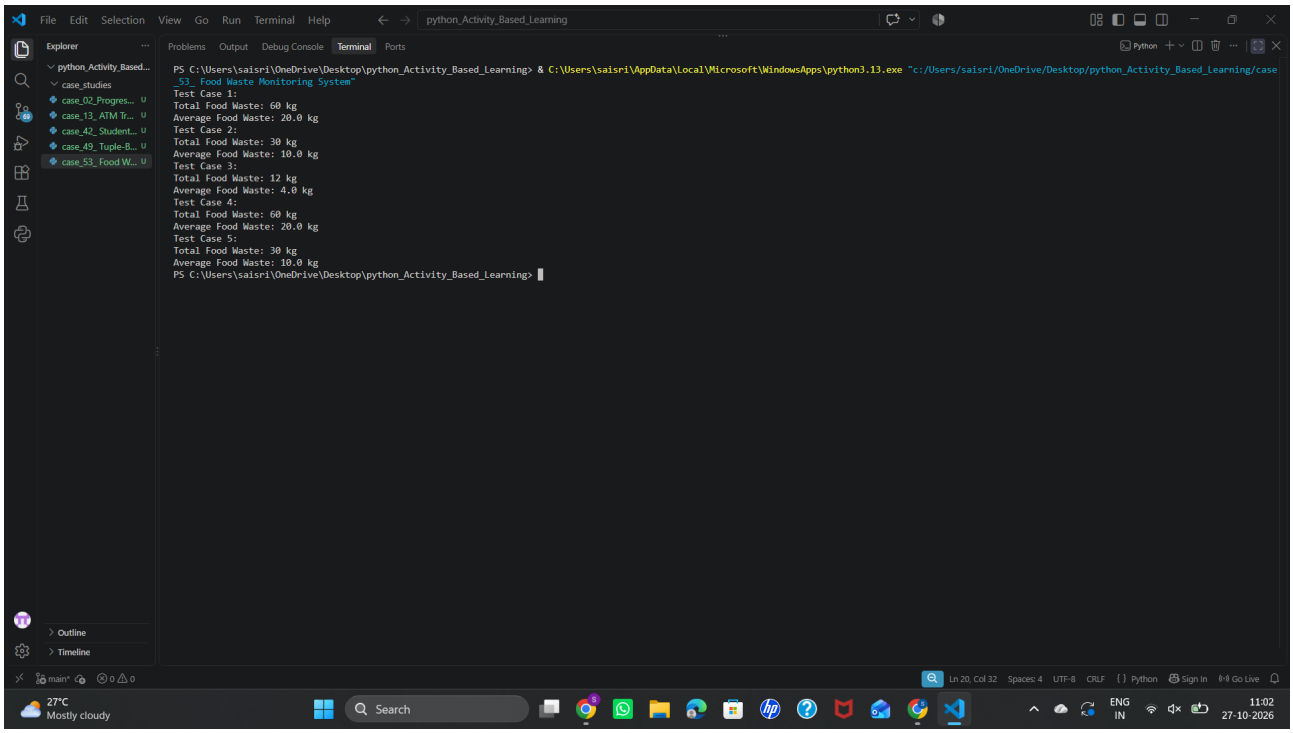
    average_waste = total_waste / len(waste_list)

    print("Test Case " + str(case_number) + ":")
    print("Total Food Waste:", total_waste, "kg")
    print("Average Food Waste:", average_waste, "kg")
    case_number = case_number + 1
```

8. Test Cases

Test	Input	Expected output
1	[15, 20, 25]	Total 60 kg, Average 20.0 kg
2	[5, 10, 15]	Total 30 kg, Average 10.0 kg
3	[2, 4, 6]	Total 12 kg, Average 4.0 kg
4	[10, 20, 30]	Total 60 kg, Average 20.0 kg
5	[8, 10, 12]	Total 30 kg, Average 10.0 kg

9. Output Screenshot



10. Conclusion

The program accumulates the daily waste with a loop and reports the total and average correctly. These figures help identify days or periods with high food wastage.

CASE 6: Smart Canteen Order and Stock Management

1. Problem Statement

Develop a Python program that processes canteen orders, calculates the bill, updates the available stock and rejects an order when the stock is not sufficient.

2. Objectives

- Read the available stock, item price and order quantity.
- Calculate the total bill for an order.
- Update the stock after a successful order.
- Reject orders that exceed the available stock.

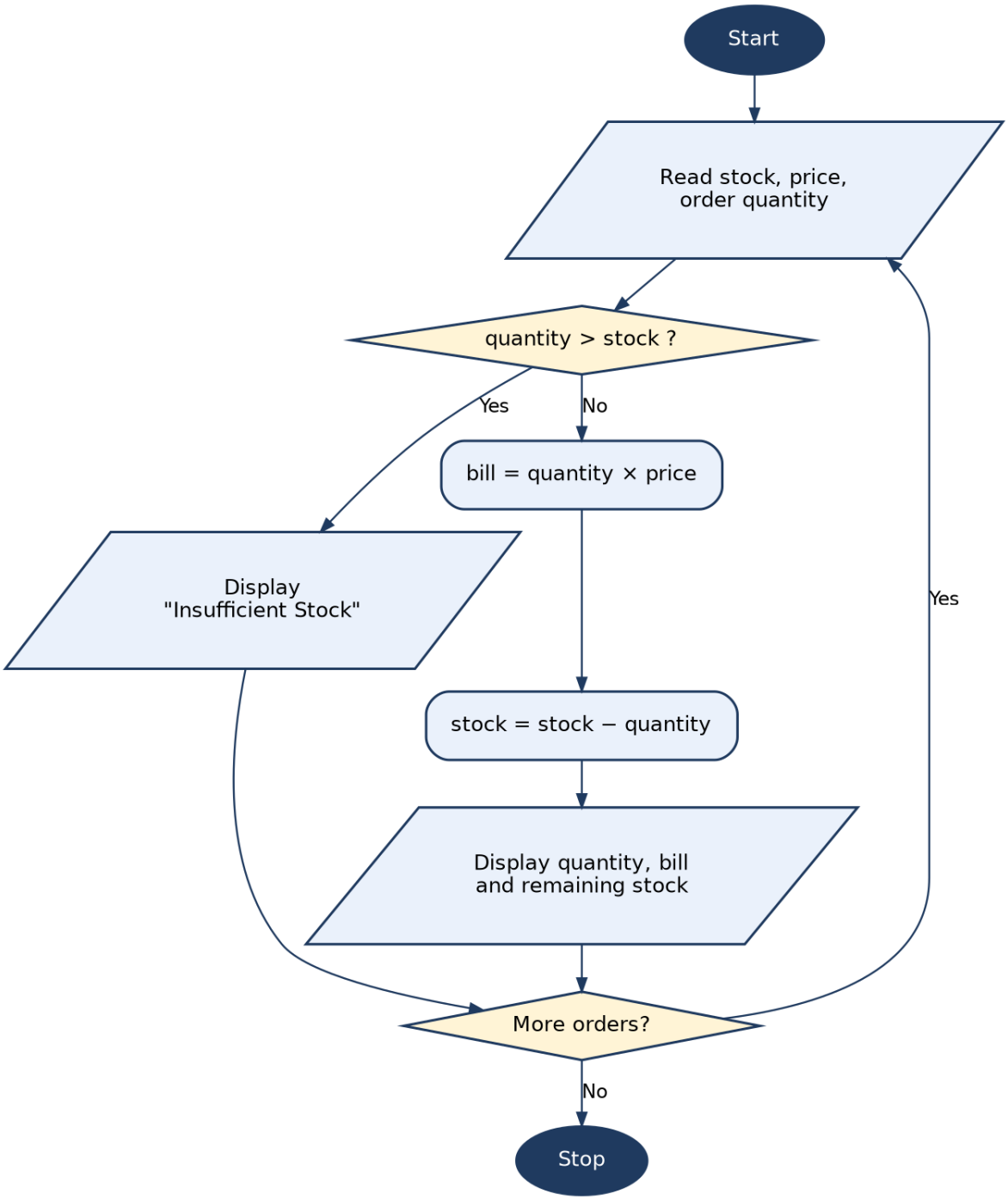
3. Assumptions

- Each test case is a tuple: (available stock, price per item, order quantity).
- Bill = order quantity × price per item.
- An order is accepted only if the quantity is not more than the stock.
- A rejected order leaves the stock unchanged.

4. Algorithm

1. Start.
2. Store the test cases as tuples (stock, price, quantity).
3. Unpack the next tuple into stock, price and quantity.
4. If quantity > stock, display Insufficient Stock.
5. Otherwise bill = quantity × price.
6. Remaining stock = stock − quantity.
7. Display the quantity, bill and remaining stock.
8. Repeat for all test cases.
9. Stop.

5. Flowchart


Start
Read stock, price, order quantity
quantity > stock ?
Yes
No
bill = quantity × price
Display "Insufficient Stock"
stock = stock − quantity
Display quantity, bill and remaining stock
More orders?
Yes
No
Stop

6. Pseudocode

```
START
STORE orders as tuples (stock, price, quantity)

FOR each order
    UNPACK order into stock, price, quantity
    IF quantity > stock THEN
        DISPLAY "Insufficient Stock"
    ELSE
        bill = quantity * price
        stock = stock - quantity
        DISPLAY quantity
        DISPLAY bill
        DISPLAY stock
    END IF
END FOR

STOP
```

7. Python Source Code / Program

```
# Case 6: Smart Canteen Order and Stock Management
# Each test case is a tuple: (available stock, price per item, order quantity)

test_cases = [
    (100, 20, 10),
    (50, 30, 5),
    (20, 15, 20),
    (10, 25, 15),
    (40, 10, 8)
]

case_number = 1
for stock, price, quantity in test_cases:       # tuple unpacking
    print("Test Case " + str(case_number) + ":")

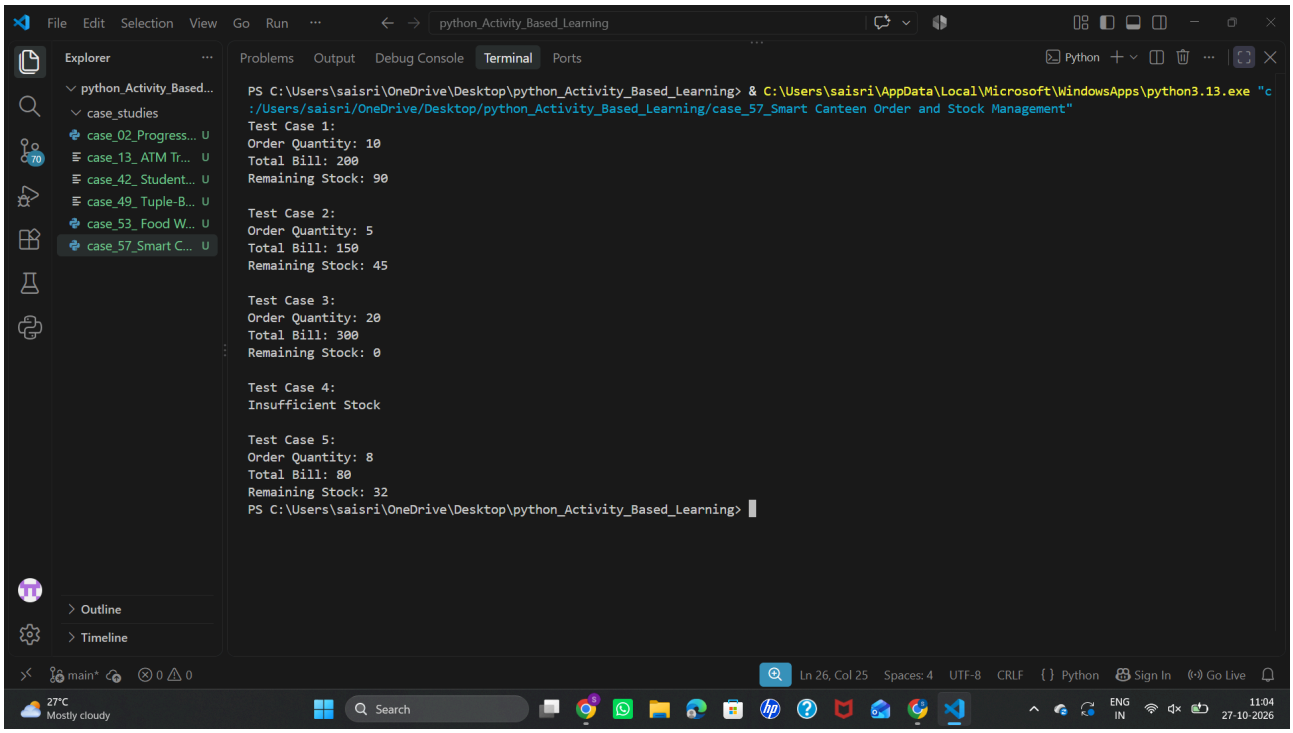
    if quantity > stock:
        print("Insufficient Stock")
    else:
        bill = quantity * price
        stock = stock - quantity
        print("Order Quantity:", quantity)
        print("Total Bill:", bill)
        print("Remaining Stock:", stock)

    if case_number < len(test_cases):
        print()
    case_number = case_number + 1
```

8. Test Cases

Test	Input	Expected output
1	Stock 100, price 20, order 10	Bill 200, Remaining 90
2	Stock 50, price 30, order 5	Bill 150, Remaining 45
3	Stock 20, price 15, order 20	Bill 300, Remaining 0
4	Stock 10, price 25, order 15	Insufficient Stock
5	Stock 40, price 10, order 8	Bill 80, Remaining 32

9. Output Screenshot



10. Conclusion

The program bills accepted orders, updates the stock and refuses orders that cannot be served. The boundary case where the whole stock is ordered (remaining 0) is handled correctly.

CASE 7: Student Academic Risk Classification

1. Problem Statement

Develop a Python program that combines lists of marks, attendance and laboratory scores to classify each student into a High, Medium or Low academic risk category.

2. Objectives

- Use parallel lists for marks, attendance and lab scores.
- Count the number of risk factors for every student.
- Classify the student according to the number of risk factors.
- Display the risk category of each student.

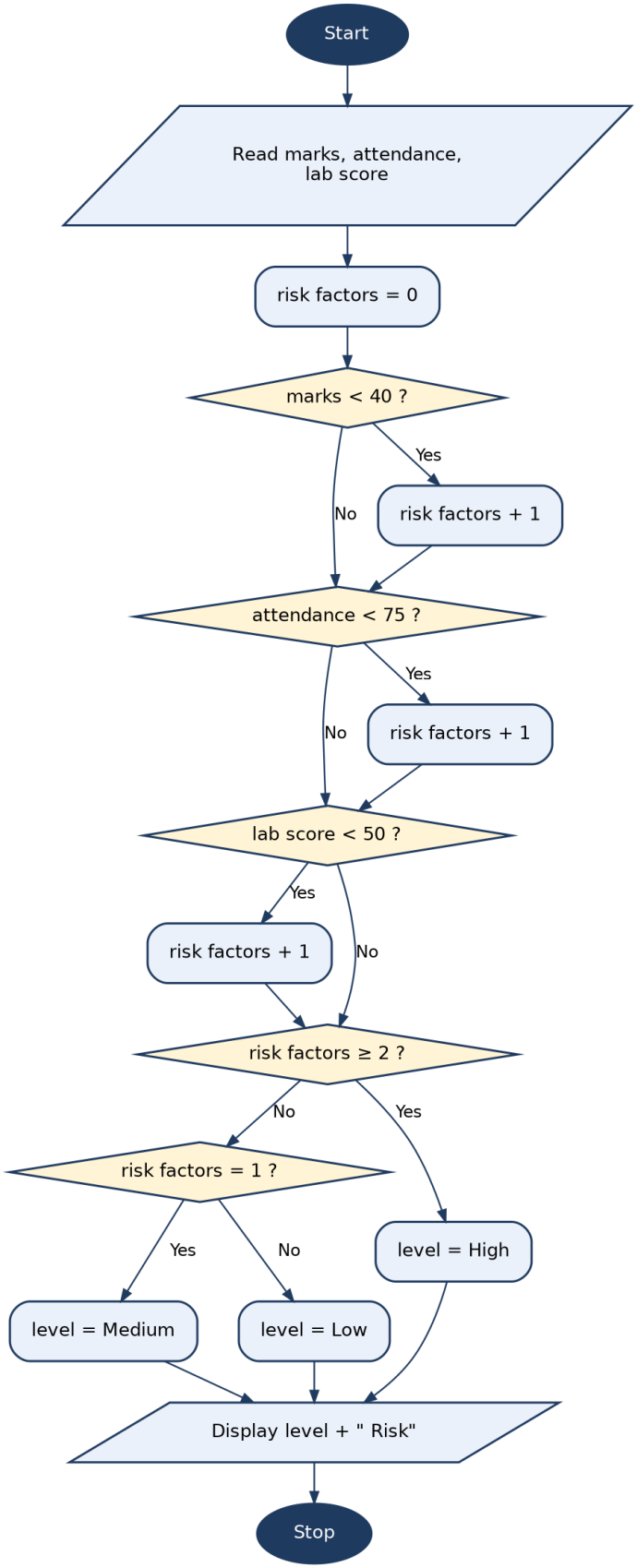
3. Assumptions

- Risk factor 1: marks below 40.
- Risk factor 2: attendance below 75%.
- Risk factor 3: laboratory score below 50.
- Two or more risk factors: High Risk; exactly one: Medium Risk; none: Low Risk.
- The three lists are of equal length and the same index refers to the same student.

4. Algorithm

1. Start.
2. Store marks, attendance and lab scores in three lists.
3. For each student set risk factors = 0.
4. Add 1 if marks < 40.
5. Add 1 if attendance < 75.
6. Add 1 if lab score < 50.
7. If risk factors ≥ 2, level = High; else if = 1, level = Medium; else Low.
8. Display the risk level.
9. Repeat for all students.
10. Stop.

5. Flowchart


Start
Read marks, attendance, lab score
risk factors = 0
marks < 40 ?
Yes
No
risk factors + 1
attendance < 75 ?
Yes
No
risk factors + 1
lab score < 50 ?
Yes
No
risk factors + 1
risk factors ≥ 2 ?
No
Yes
risk factors = 1 ?
Yes
No
level = High
level = Medium
level = Low
Display level + " Risk"
Stop

6. Pseudocode

```
START
STORE marks, attendance, lab_scores in lists

FOR each student index i
    risk = 0
    IF marks[i] < 40 THEN risk = risk + 1
    IF attendance[i] < 75 THEN risk = risk + 1
    IF lab_scores[i] < 50 THEN risk = risk + 1

    IF risk >= 2 THEN
        level = "High"
    ELSE IF risk == 1 THEN
        level = "Medium"
    ELSE
        level = "Low"
    END IF
    DISPLAY level, "Risk"
END FOR

STOP
```

7. Python Source Code / Program

```
# Case 7: Student Academic Risk Classification
# Risk factors: marks < 40, attendance < 75, lab score < 50

marks = [35, 55, 70, 80, 60]
attendance = [60, 70, 85, 90, 80]
lab_scores = [40, 80, 75, 90, 60]

for i in range(len(marks)):
    risk_factors = 0

    if marks[i] < 40:
        risk_factors = risk_factors + 1
    if attendance[i] < 75:
        risk_factors = risk_factors + 1
    if lab_scores[i] < 50:
        risk_factors = risk_factors + 1

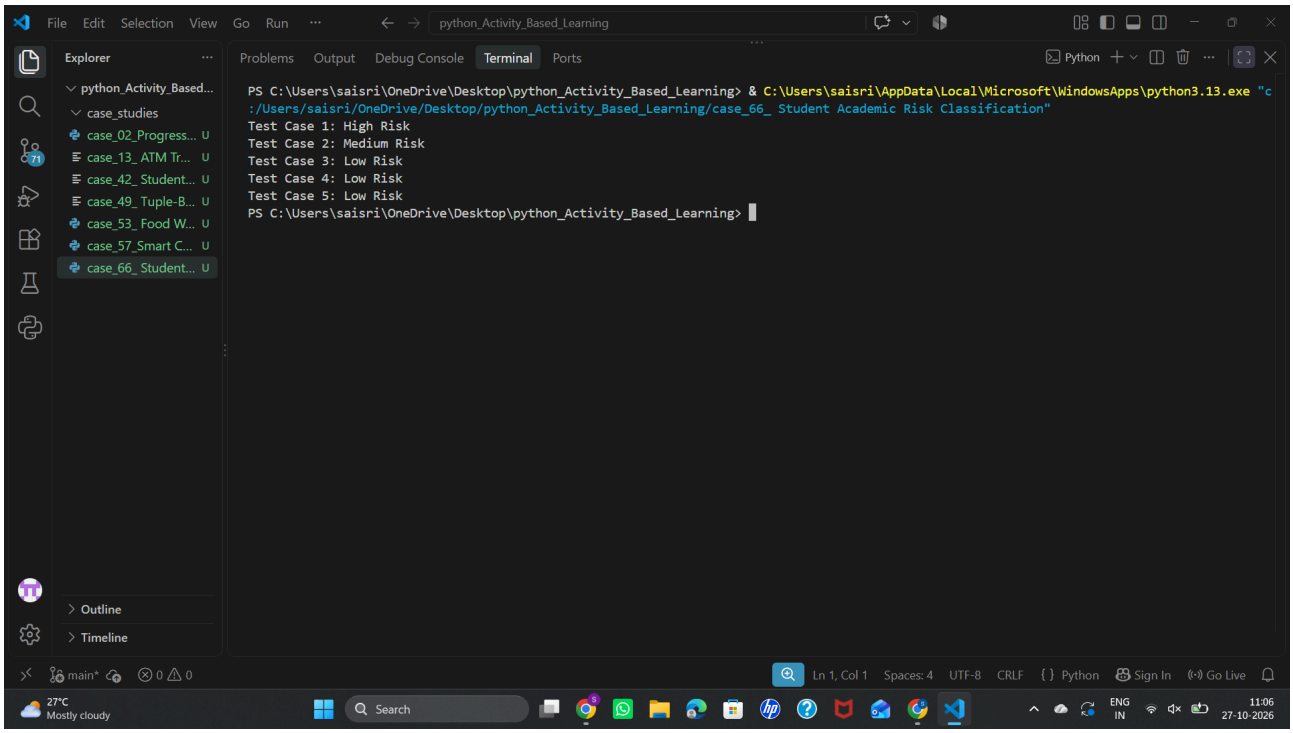
    if risk_factors >= 2:
        level = "High"
    elif risk_factors == 1:
        level = "Medium"
    else:
        level = "Low"

    print("Test Case " + str(i + 1) + ":", level, "Risk")
```

8. Test Cases

Test	Input	Expected output
1	Marks 35, Att 60, Lab 40	High Risk
2	Marks 55, Att 70, Lab 80	Medium Risk
3	Marks 70, Att 85, Lab 75	Low Risk
4	Marks 80, Att 90, Lab 90	Low Risk
5	Marks 60, Att 80, Lab 60	Low Risk

9. Output Screenshot



10. Conclusion

By counting simultaneous risk factors, the program separates students needing urgent support from those who are safe. The results match the stated classification rules for all test cases.

CASE 8: Smart Electricity Billing for Multiple Apartments

1. Problem Statement

Develop a Python program that calculates the electricity bill of several apartments using progressive tariff slabs and a fixed charge, and displays the units and bill of each apartment.

2. Objectives

- Store the consumption of several apartments in a list.
- Apply progressive slab rates to each apartment's units.
- Add the fixed charge to get the final bill.
- Display the units and bill apartment-wise.

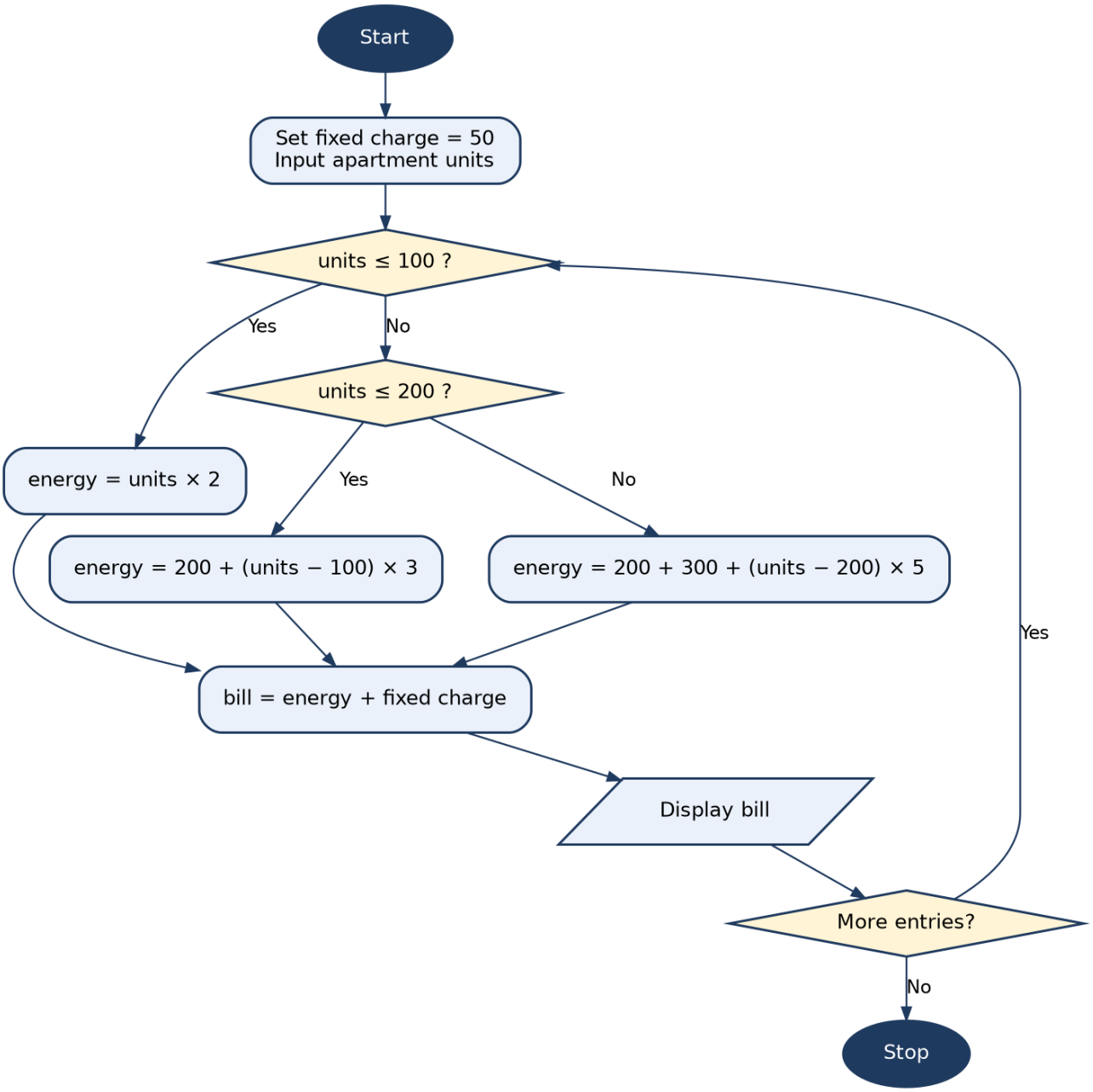
3. Assumptions

- These are sample tariff rules; they are not official rates.
- First 100 units at ₹2, units 101 to 200 at ₹3, units above 200 at ₹5.
- A fixed charge of ₹50 applies to every apartment.
- Apartments are numbered from 1 in the order of the list.

4. Algorithm

1. Start.
2. Set the fixed charge to 50 and store apartment units in a list.
3. Take the next apartment's units.
4. If units ≤ 100, energy = units × 2.
5. Else if units ≤ 200, energy = 200 + (units − 100) × 3.
6. Else energy = 500 + (units − 200) × 5.
7. Bill = energy + fixed charge.
8. Display the apartment number, units and bill.
9. Repeat for all apartments.
10. Stop.

5. Flowchart


Start
Set fixed charge = 50
Input apartment units
units ≤ 100 ?
Yes
No
units ≤ 200 ?
energy = units × 2
Yes
No
energy = 200 + (units − 100) × 3
energy = 200 + 300 + (units − 200) × 5
bill = energy + fixed charge
Display bill
More entries?
Yes
No
Stop

6. Pseudocode

```
START
SET fixed_charge = 50
STORE apartment units in a list

apartment_number = 1
FOR each units in the list
    IF units <= 100 THEN
        energy = units * 2
    ELSE IF units <= 200 THEN
        energy = 200 + (units - 100) * 3
    ELSE
        energy = 500 + (units - 200) * 5
    END IF

    bill = energy + fixed_charge
    DISPLAY apartment_number, units, bill
    apartment_number = apartment_number + 1
END FOR

STOP
```

7. Python Source Code / Program

```
# Case 8: Smart Electricity Billing for Multiple Apartments
# Slabs: 0-100 units @ Rs.2, 101-200 units @ Rs.3, above 200 units @ Rs.5
# Fixed charge: Rs.50 per apartment

fixed_charge = 50
apartment_units = [50, 100, 150, 250, 300]

apartment_number = 1
for units in apartment_units:
    if units <= 100:
        energy_charge = units * 2
    elif units <= 200:
        energy_charge = 100 * 2 + (units - 100) * 3
    else:
        energy_charge = 100 * 2 + 100 * 3 + (units - 200) * 5

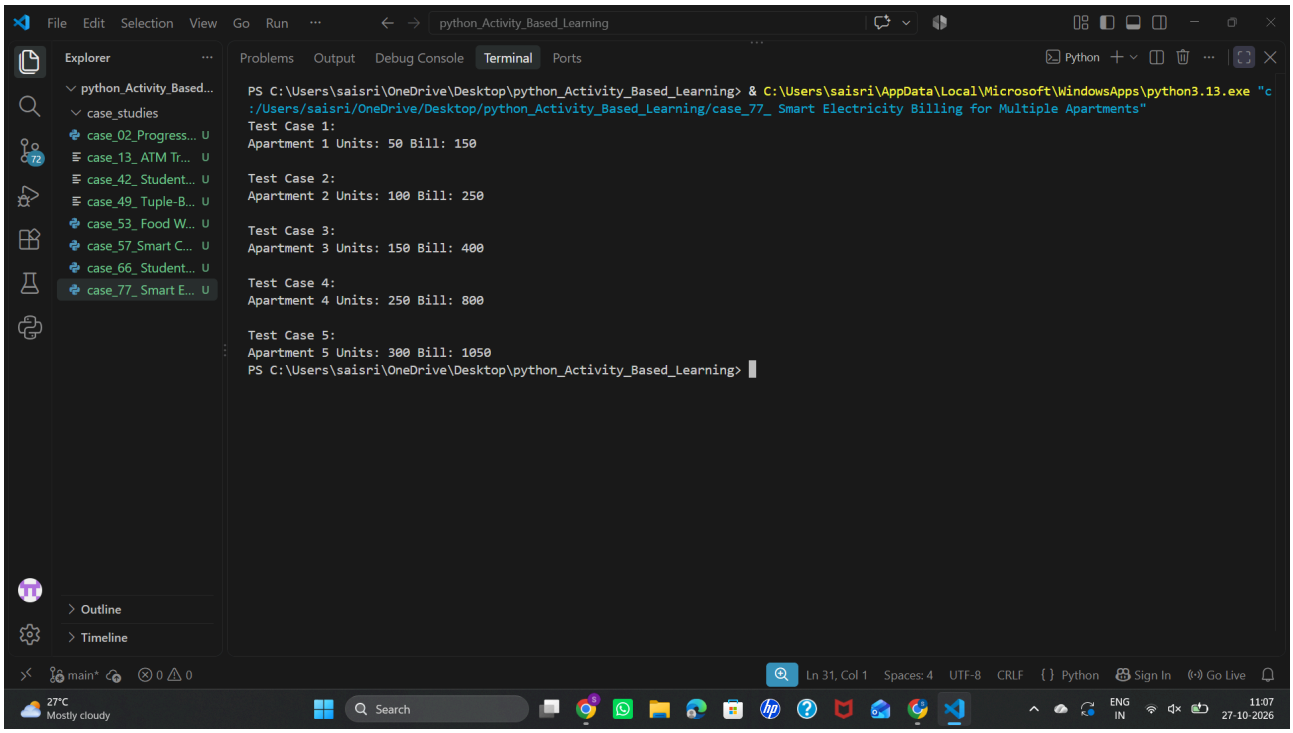
    bill = energy_charge + fixed_charge

    print("Test Case " + str(apartment_number) + ":")
    print("Apartment", apartment_number, "Units:", units, "Bill:", bill)
    if apartment_number < len(apartment_units):
        print()
    apartment_number = apartment_number + 1
```

8. Test Cases

Test	Input	Expected output
1	Apartment 1: 50 units	Bill 150
2	Apartment 2: 100 units	Bill 250
3	Apartment 3: 150 units	Bill 400
4	Apartment 4: 250 units	Bill 800
5	Apartment 5: 300 units	Bill 1050

9. Output Screenshot



10. Conclusion

The program bills every apartment fairly under the same progressive tariff. Apartments with higher consumption pay a larger share per unit, which encourages saving electricity.

CASE 9: Examination Question Difficulty and Performance Analysis

1. Problem Statement

Develop a Python program that compares the answer strings of several students with an answer key, calculates the question-wise success rate, classifies each question's difficulty and identifies the most frequently missed question.

2. Objectives

- Compare each student's answer string with the answer key.
- Count correct and wrong answers for every question.
- Calculate the success rate and classify question difficulty.
- Identify the most frequently missed question.

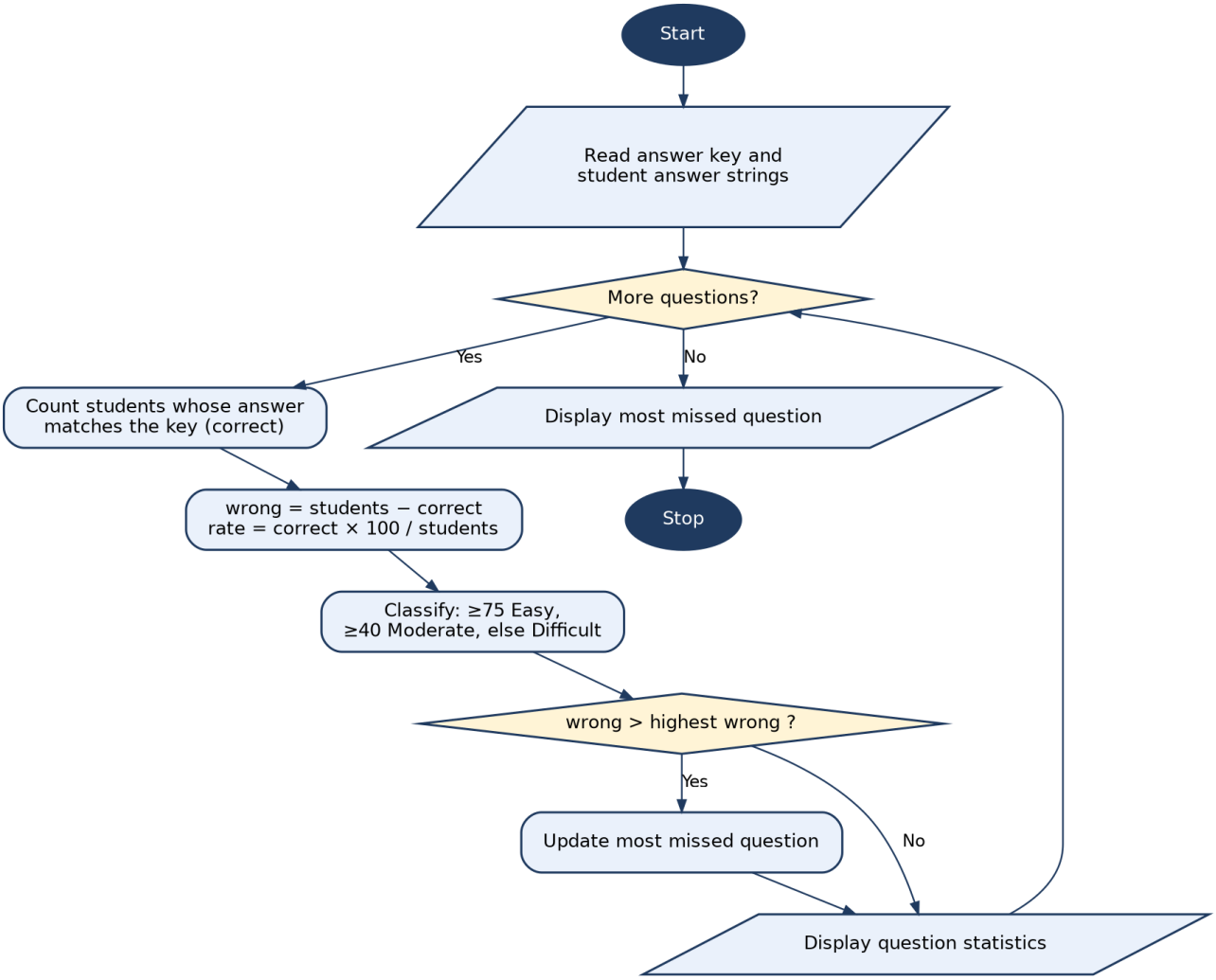
3. Assumptions

- Each student answer is a string with one letter per question.
- Success rate = correct answers × 100 ÷ number of students.
- Success rate ≥ 75: Easy; 40 to 74: Moderate; below 40: Difficult.
- If two questions are missed equally often, the first one is reported.

4. Algorithm

1. Start.
2. Store the answer key and the list of student answer strings.
3. Display the total number of students.
4. For each question position, count the students whose answer equals the key.
5. wrong = students − correct; success rate = correct × 100 ÷ students.
6. Classify the question as Easy, Moderate or Difficult.
7. If wrong is greater than the highest wrong so far, remember this question.
8. Display the question statistics.
9. After all questions, display the most missed question.
10. Stop.

5. Flowchart


Start
Read answer key and student answer strings
More questions?
Yes
No
Count students whose answer matches the key (correct)
Display most missed question
wrong = students − correct
rate = correct × 100 / students
Stop
Classify: ≥75 Easy, ≥40 Moderate, else Difficult
wrong > highest wrong ?
Yes
No
Update most missed question
Display question statistics

6. Pseudocode

```
START
STORE answer_key and student answers
DISPLAY total students

highest_wrong = -1
FOR each question position q
    correct = 0
    FOR each student answer
        IF answer[q] == answer_key[q] THEN
            correct = correct + 1
        END IF
    END FOR

    wrong = total_students - correct
    success_rate = correct * 100 / total_students

    IF success_rate >= 75 THEN difficulty = "Easy"
    ELSE IF success_rate >= 40 THEN difficulty = "Moderate"
    ELSE difficulty = "Difficult"

    IF wrong > highest_wrong THEN
        highest_wrong = wrong
        most_missed = q + 1
    END IF
    DISPLAY correct, wrong, success_rate, difficulty
END FOR

DISPLAY most_missed
STOP
```

7. Python Source Code / Program

```
# Case 9: Examination Question Difficulty and Performance Analysis

answer_key = "ABCD"
students = ["ABCD", "ABCD", "ABCD", "BCDA", "BCDA"]

total_students = len(students)
print("Total Students:", total_students)

most_missed = 1
highest_wrong = -1

for q in range(len(answer_key)):
    correct = 0
    for answer in students:
        if answer[q] == answer_key[q]:
            correct = correct + 1

    wrong = total_students - correct
    success_rate = (correct * 100) / total_students

    if success_rate >= 75:
        difficulty = "Easy"
    elif success_rate >= 40:
        difficulty = "Moderate"
    else:
        difficulty = "Difficult"

    if wrong > highest_wrong:
        highest_wrong = wrong
        most_missed = q + 1

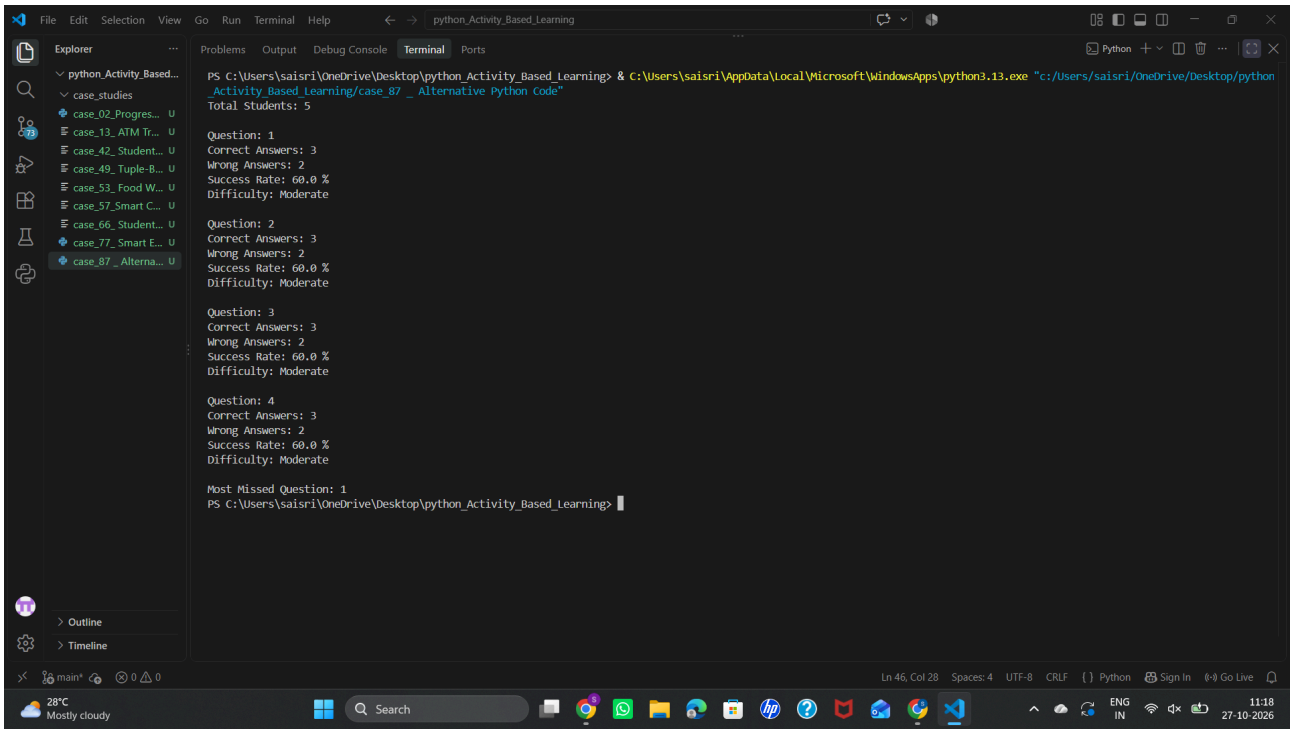
    print()
    print("Question:", q + 1)
    print("Correct Answers:", correct)
    print("Wrong Answers:", wrong)
    print("Success Rate:", success_rate, "%")
    print("Difficulty:", difficulty)

print()
print("Most Missed Question:", most_missed)
```

8. Test Cases

Test	Input	Expected output
1	Key ABCD; 5 students (ABCD ×3, BCDA ×2); Question 1	Correct 3, Wrong 2, 60.0 %, Moderate
2	Same data; Question 2	Correct 3, Wrong 2, 60.0 %, Moderate
3	Same data; Question 3	Correct 3, Wrong 2, 60.0 %, Moderate
4	Same data; Question 4	Correct 3, Wrong 2, 60.0 %, Moderate
5	All questions missed equally (tie)	Most Missed Question: 1

9. Output Screenshot



10. Conclusion

The program turns raw answer strings into useful question-level statistics. When several questions are missed equally, the first one is reported, so the result is always predictable.

CASE 10: Advanced String-Based Registration Record Analyzer

1. Problem Statement

Develop a Python program that parses registration strings of the form ID-Name-Year, validates each field, detects duplicate IDs and reports an error classification for every record.

2. Objectives

- Split a record string into its fields.
- Validate the ID, name and year fields.
- Detect duplicate registration IDs.
- Classify every record as valid or with a specific error.

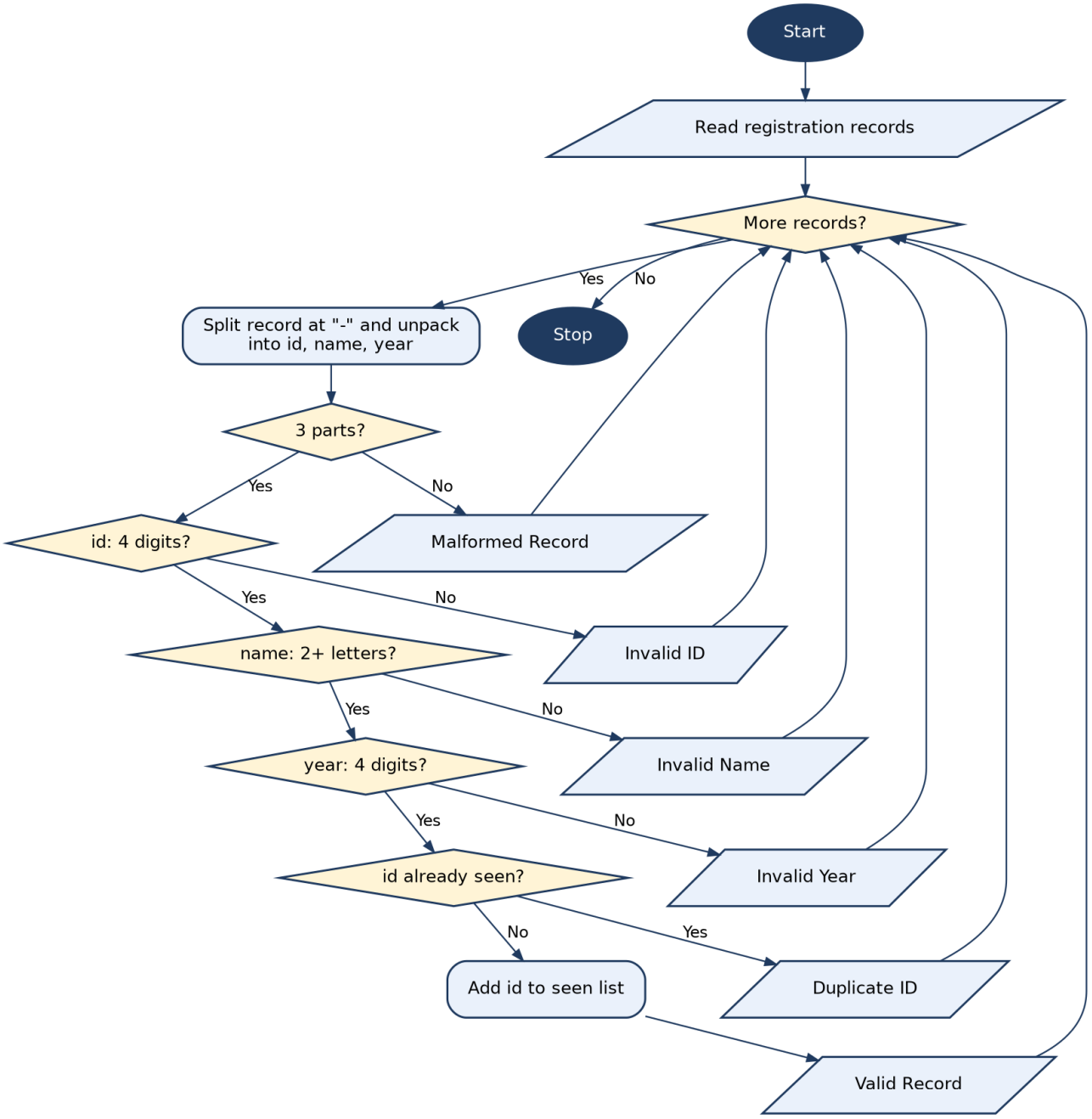
3. Assumptions

- A record has the format ID-Name-Year, for example 1234-Ravi-2026.
- A valid ID has exactly 4 digits.
- A valid name has at least 2 letters and only letters.
- A valid year has exactly 4 digits.
- IDs of valid records are remembered; a later record with the same ID is a duplicate.

4. Algorithm

1. Start.
2. Store the test records and an empty list of seen IDs.
3. Take the next record and split it at the “-” character.
4. If it does not have exactly 3 parts, report Malformed Record.
5. Unpack the parts into id, name and year.
6. If the ID is not 4 digits, report Invalid ID.
7. Else if the name is shorter than 2 letters or not alphabetic, report Invalid Name.
8. Else if the year is not 4 digits, report Invalid Year.
9. Else if the ID is already in the seen list, report Duplicate ID.
10. Otherwise add the ID to the seen list and report Valid Record.
11. Repeat for all records.
12. Stop.

5. Flowchart


Start
Read registration records
More records?
Yes
No
Stop
Split record at "-" and unpack into id, name, year
3 parts?
Yes
No
Malformed Record
id: 4 digits?
Yes
No
Invalid ID
name: 2+ letters?
Yes
No
Invalid Name
year: 4 digits?
Yes
No
Invalid Year
id already seen?
No
Yes
Add id to seen list
Duplicate ID
Valid Record

6. Pseudocode

```
START
seen_ids = empty list

FOR each record
    parts = SPLIT record at "-"
    IF number of parts != 3 THEN
        DISPLAY record, "Malformed Record"
    ELSE
        UNPACK parts into id, name, year
        IF length of id != 4 OR id is not all digits THEN
            result = "Invalid ID"
        ELSE IF length of name < 2 OR name is not all letters THEN
            result = "Invalid Name"
        ELSE IF length of year != 4 OR year is not all digits THEN
            result = "Invalid Year"
        ELSE IF id is in seen_ids THEN
            result = "Duplicate ID"
        ELSE
            ADD id to seen_ids
            result = "Valid Record"
        END IF
        DISPLAY record, result
    END IF
END FOR

STOP
```

7. Python Source Code / Program

```
Case 10.0:
# Case 10.0: Advanced String-Based Registration Record Analyzer
# Record format: ID-Name-Year   (e.g. 1234-Ravi-2026)

test_cases = [
    ["1234-Ravi-2026"],
    ["12-Ravi-2026"],
    ["1235-A-2026"],
    ["1236-Sita-20A6"],
    ["1237-Raju-2026", "1237-Kiran-2025"]
]

seen_ids = []
case_number = 1

for records in test_cases:
    print("Test Case " + str(case_number) + ":")

    for record in records:
        parts = record.split("-")
        if len(parts) != 3:
            print(record, "Malformed Record")
            continue

        student_id, name, year = parts            # tuple/list unpacking

        if len(student_id) != 4 or not student_id.isdigit():
            result = "Invalid ID"
        elif len(name) < 2 or not name.isalpha():
            result = "Invalid Name"
        elif len(year) != 4 or not year.isdigit():
            result = "Invalid Year"
        elif student_id in seen_ids:
            result = "Duplicate ID"
        else:
            seen_ids.append(student_id)
            result = "Valid Record"

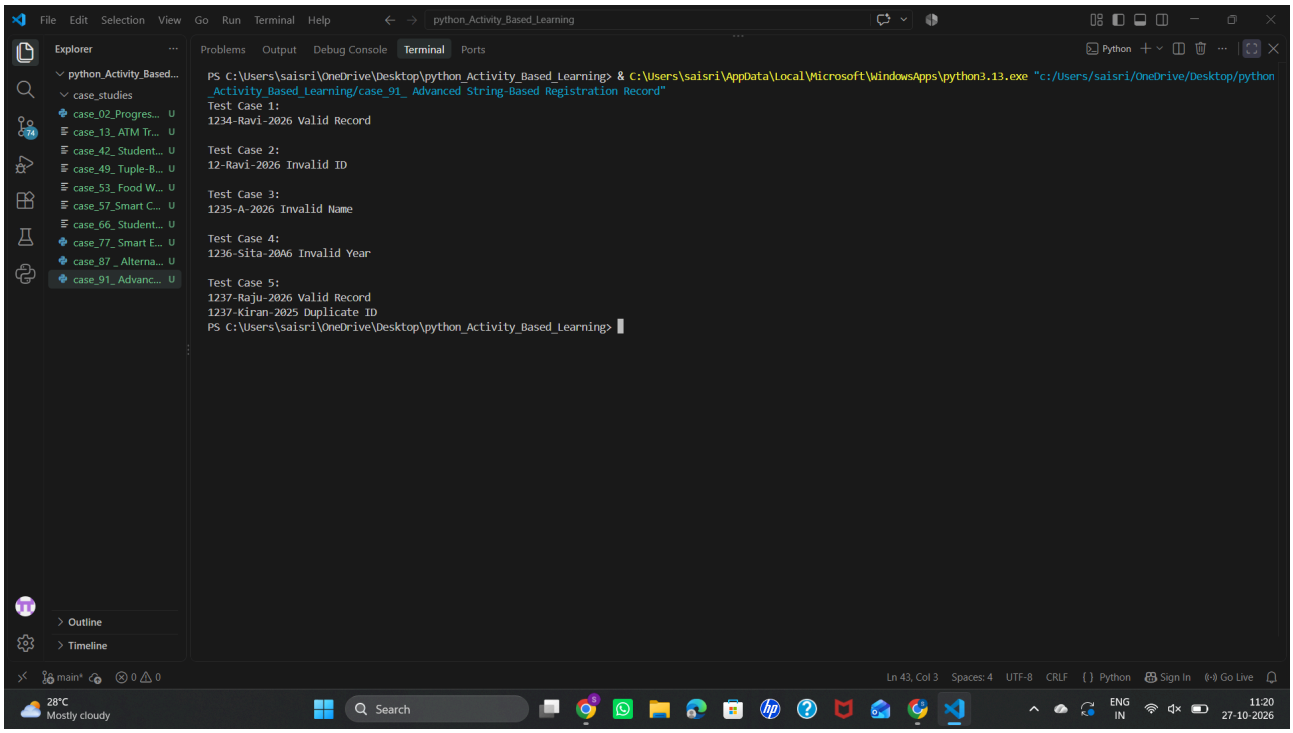
        print(record, result)

    if case_number < len(test_cases):
        print()
    case_number = case_number + 1
```

8. Test Cases

Test	Input	Expected output
1	1234-Ravi-2026	Valid Record
2	12-Ravi-2026	Invalid ID
3	1235-A-2026	Invalid Name
4	1236-Sita-20A6	Invalid Year
5	1237-Raju-2026, then 1237-Kiran-2025	Valid Record, then Duplicate ID

9. Output Screenshot



10. Conclusion

String methods such as split(), isdigit() and isalpha() make field-level validation straightforward. The analyzer gives a specific error for each faulty record and also catches duplicate IDs.